Nevil C Prince
Email: ep23btech11019@iith.ac.in
Course: Multi-Agent Systems (MAS)
Instructor: Dr. Venkatraman Renganathan

Multi-Agent Systems (MAS) – Assignment #4
Roll No: EP23BTECH11019
Due: October 09, 2026

Problem 1

Data-Driven Epidemic Modelling and Vaccination on Networks. A population of $N = 300$ individuals is connected through an undirected contact network (`network.csv`), and the SIR state of every individual is recorded daily (`epidemic_data.csv`). From these data: (1) compute and plot $S[k], I[k], R[k]$ and report the peak, final size and infected fraction; (2) estimate γ; (3) estimate β and R_0; (4) simulate the fitted continuous SIR model and compare it with the data; (5) decide whether the outbreak was an epidemic; (6) compare random, degree-based and eigenvector-centrality vaccination with a budget of $B = 15$ nodes.

Solution: All the numbers below come from the code at the end of this problem (it is also in the notebook `problem1.ipynb`). The data cover 101 days ($k = 0, \ldots, 100$) and the network has 1558 edges.

Part 1: Observed trajectories. For each day I simply counted how many people were in each state. The starting point is $S[0] = 296$, $I[0] = 4$, $R[0] = 0$.

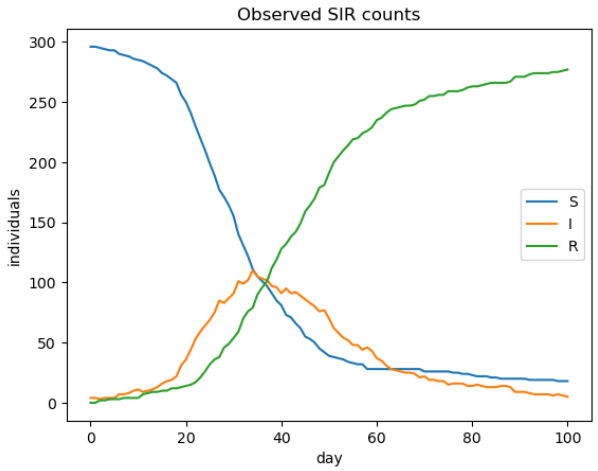


Figure 1: Observed $S[k]$, $I[k]$ and $R[k]$.

(a) Maximum number of simultaneously infected individuals: $\max_k I[k] = \mathbf{110}$.

(b) The peak occurs on day **34**.

(c) Individuals who experienced infection: $N - S[100] = 300 - 18 = \mathbf{282}$.

(d) Fraction of the population infected: $282/300 = \mathbf{0.94}$.

Part 2: Recovery rate. I averaged the forward-Euler estimate $\gamma_k = (R[k+1] - R[k])/(I[k]\Delta t)$ over all days with $I[k] > 0$:

$$\hat{\gamma} = \frac{1}{|\mathcal{K}|} \sum_{k \in \mathcal{K}} \frac{R[k+1] - R[k]}{I[k]\Delta t} \approx \mathbf{0.0713\ day^{-1}}.$$

The average infectious duration is $1/\hat{\gamma} \approx \mathbf{14.0}$ days.

Part 3: Transmission rate and reproduction number. The susceptible equation $\dot{S} = -\beta SI$ discretizes to $S[k+1] - S[k] \approx -\beta S[k]I[k]\Delta t$, so averaging over $\mathcal{K}_\beta = \{k : S[k]I[k] > 0\}$ gives

$$\hat{\beta} = \frac{1}{|\mathcal{K}_\beta|} \sum_{k \in \mathcal{K}_\beta} \frac{S[k] - S[k+1]}{S[k]I[k]\Delta t} \approx \mathbf{7.97 \times 10^{-4}} \text{ (per person per day).}$$

The model works with head-counts, so the reproduction number is

$$\hat{R}_0 = \frac{\hat{\beta}\, S(0)}{\hat{\gamma}} \approx \frac{\hat{\beta} N}{\hat{\gamma}} \approx \mathbf{3.35}.$$

Since $\hat{R}_0 > 1$, an infected person passes the disease to more than one other person early on, so the infection grows at first and only dies down once enough people have left the susceptible pool. So yes, this disease can cause an epidemic.

Part 4: Fitted SIR model. I integrated $\dot{S} = -\hat{\beta}SI$, $\dot{I} = \hat{\beta}SI - \hat{\gamma}I$, $\dot{R} = \hat{\gamma}I$ from $(S, I, R)(0) = (296, 4, 0)$.

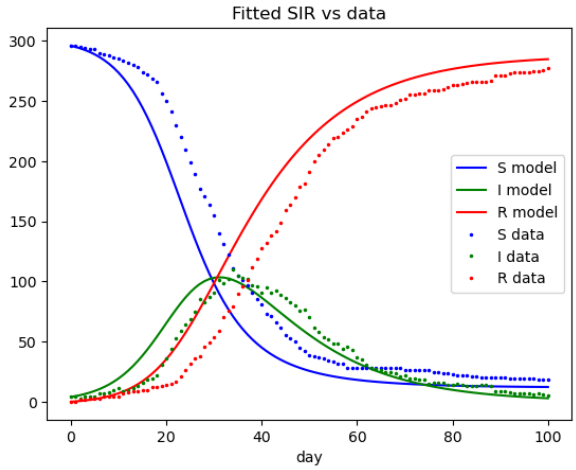


Figure 2: Simulated SIR model (lines) against the observed data (dots).

The model gets the overall shape right: a rise, a single peak and then a burnout. The details are a bit off. It peaks at about 103 infected on day 31 (data: 110 on day 34) and leaves about 12 susceptibles at the end (data: 18). The RMSE is roughly 26, 12 and 24 individuals for S, I and R. That is not surprising, because the model assumes everyone mixes with everyone while the real spread happens along a sparse network, and $\hat{\beta}, \hat{\gamma}$ come from simple Euler ratios.

Part 5: Was it an epidemic? Yes, it was an epidemic. The evidence points the same way everywhere:

- the infected count grew from 4 to 110 (about 27×) before declining;
- $\hat{R}_0 \approx 3.35 > 1$, so early growth is expected;
- at the peak 37% of the population was infected at the same moment;
- 94% of the population ended up being infected;
- the fitted SIR model shows the same behaviour with no special assumptions.

Part 6: Vaccination with $B = 15$. For this part I ran a stochastic, discrete-time SIR process on the actual network. Each day a susceptible node is infected with probability $1 - (1-p)^m$, where m is its number of infected neighbours, and an infected node recovers with probability $\hat{\gamma}$. I calibrated the edge probability p so that the unvaccinated final size matches the data ($p = 0.03$; using the raw edge weight 0.0963 overshoots to about 299 infected). Vaccinated nodes are removed before day 0. Every row below is an average over 300 runs, and N_{infected} does not count the 15 vaccinated nodes.

Strategy	$I_{\max}$	T_{peak} (day)	N_{infected}
No vaccination	122.0	27.3	284.6
Random	111.8	28.8	268.3
Degree-based	**101.1**	31.4	**263.2**
Eigenvector-based	102.5	31.7	265.0

Table 1: Comparison of vaccination strategies (mean of 300 simulations).

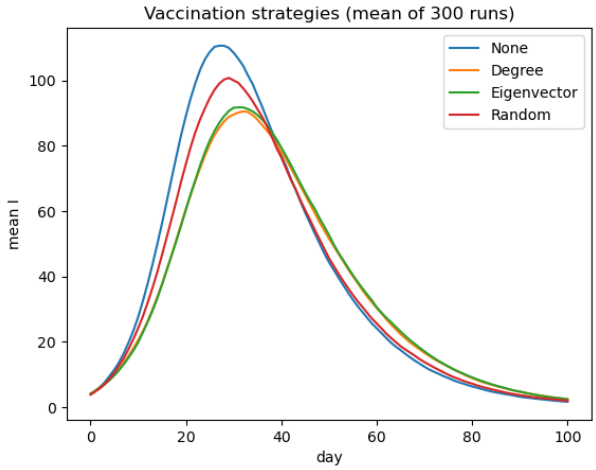


Figure 3: Mean infection trajectories for each strategy.

(a) **Best strategy.** Degree-based and eigenvector-based vaccination did best, and they are almost tied. Both bring the peak down from 122 to about 101–103 and push it back by roughly four to five days. Random vaccination helps less ($I_{\max} \approx 112$). The final sizes of all three strategies are close, which makes sense: 15 people are only 5% of the population and the run-to-run spread in N_{infected} is several individuals, so the peak is where the difference is clearest.

(b) **Why some nodes matter more.** A hub has many neighbours, so it would have passed the infection along many edges, and removing it cuts all of those paths at once. A random node has only about 10 edges on average, while the top nodes have 16–22. Taking out hubs also lowers the largest eigenvalue of the network ($\lambda_{\max}(A) \approx 11.4$ before removal), which largely decides how easily an outbreak takes off. Eigenvector centrality additionally favours nodes whose neighbours are well connected too, so it goes after the dense core.

(c) **Degree versus eigenvector centrality.** No, not always. Here only 11 of the 15 chosen nodes are the same. A node with high degree whose neighbours are poorly connected gets a lower eigenvector score, while a node of moderate degree sitting in a cluster of hubs gets a higher one. In this network the two rankings are strongly correlated (degrees are fairly even, with a mean of 10.4 and a maximum of 22), which is why the results are nearly identical. On networks with strong hubs or clear communities the two can differ a lot.

Source code for Problem 1 (also in the notebook ***problem1.ipynb****).*

```
import csv

import matplotlib.pyplot as plt
import numpy as np
from scipy.integrate import solve_ivp

with open("epidemic_data.csv") as f:
    rows = list(csv.DictReader(f))
N = 300
```

```
K = max(int(r["day"]) for r in rows) + 1
st = np.zeros((K, N), dtype="U1")  # st[day, node] = 'S'/'I'/'R'
for r in rows:
    st[int(r["day"]), int(r["node"]) - 1] = r["state"]

A = np.zeros((N, N))  # symmetric adjacency (undirected)
with open("network.csv") as f:
    edges = list(csv.DictReader(f))
for r in edges:
    i, j = int(r["source"]) - 1, int(r["target"]) - 1
    A[i, j] = A[j, i] = 1.0
print("days", K, "edges", int(A.sum() / 2))

S = (st == "S").sum(1)
I = (st == "I").sum(1)
R = (st == "R").sum(1)
k = np.arange(K)
plt.plot(k, S, label="S")
plt.plot(k, I, label="I")
plt.plot(k, R, label="R")
plt.xlabel("day")
plt.ylabel("individuals")
plt.legend()
plt.title("Observed SIR counts")
plt.show()
print("(a) max infected      :", I.max())
print("(b) day of max        :", I.argmax())
print("(c) ever infected     :", N - S[-1])
print("(d) fraction infected :", (N - S[-1]) / N)

Kg = [t for t in range(K - 1) if I[t] > 0]
gamma = np.mean([(R[t + 1] - R[t]) / I[t] for t in Kg])
print("gamma =", gamma, "| mean infectious duration =", 1 / gamma, "days")

Kb = [t for t in range(K - 1) if S[t] * I[t] > 0]
beta = np.mean([(S[t] - S[t + 1]) / (S[t] * I[t]) for t in Kb])
R0 = beta * N / gamma
print("beta =", beta, "| R0 =", R0)

def sir(t, y):
    s, i, r = y
    return [-beta * s * i, beta * s * i - gamma * i, gamma * i]


sol = solve_ivp(sir, [0, K - 1], [S[0], I[0], R[0]], t_eval=k, rtol=1e-8)
for y, c, n in zip(sol.y, "bgr", "SIR", strict=True):
    plt.plot(k, y, c + "-", label=n + " model")
for y, c, n in zip((S, I, R), "bgr", "SIR", strict=True):
    plt.plot(k, y, c + ".", ms=3, label=n + " data")
plt.xlabel("day")
plt.legend()
plt.title("Fitted SIR vs data")
plt.show()
print("RMSE (S, I, R):", np.sqrt(np.mean((sol.y - np.array([S, I, R])) ** 2, 1)))
print("model peak:", sol.y[1].max(), "on day", sol.y[1].argmax(), "| model final S:", sol
    .y[0][-1])

def simulate(p, vacc, rng):
    s = (st[0] == "S").copy()
    i = (st[0] == "I").copy()
    v = np.zeros(N, bool)
    v[list(vacc)] = True
    s &= ~v
    i &= ~v
    Ik = [i.sum()]
```

```
    for _ in range(K - 1):
        n_inf = A @ i.astype(float)
        new = s & (rng.random(N) < 1 - (1 - p) ** n_inf)
        rec = i & (rng.random(N) < gamma)
        s = s & ~new
        i = (i & ~rec) | new
        Ik.append(i.sum())
    return np.array(Ik), s.sum()


rng = np.random.default_rng(0)
target = N - S[-1]
ps = np.linspace(0.005, 0.2, 40)
fs = [np.mean([N - simulate(p, [], rng)[1] for _ in range(30)]) for p in ps]
p = ps[np.argmin(np.abs(np.array(fs) - target))]
print("calibrated p =", p)

B = 15
deg = A.sum(1)
evals, evecs = np.linalg.eigh(A)
ev = np.abs(evecs[:, -1])  # dominant eigenvector
top_deg = np.argsort(-deg)[:B]
top_eig = np.argsort(-ev)[:B]
print(
    "lambda_max =",
    evals[-1],
    "| mean degree =",
    deg.mean(),
    "| overlap of top-15 sets:",
    len(set(top_deg) & set(top_eig)),
)

strategies = {"None": [], "Degree": top_deg, "Eigenvector": top_eig, "Random": None}
curves = {}
for name, vac in strategies.items():
    runs, nvac = [], (B if vac is None else len(vac))
    for _ in range(300):
        v = rng.choice(N, B, replace=False) if vac is None else vac
        runs.append(simulate(p, v, rng))
    Ia = np.array([r[0] for r in runs])
    n_inf = np.array([N - r[1] - nvac for r in runs])  # excludes vaccinated nodes
    curves[name] = Ia.mean(0)
    print(
        f"{name:12s} Imax={Ia.max(1).mean():6.1f}  "
        f"Tpeak={Ia.argmax(1).mean():5.1f}  Ninfected={n_inf.mean():6.1f}"
    )

for n, c in curves.items():
    plt.plot(c, label=n)
plt.xlabel("day")
plt.ylabel("mean I")
plt.legend()
plt.title("Vaccination strategies (mean of 300 runs)")
plt.show()
```

Problem 2

Temporal Logic Specification for a Cyber-security System. For an online authentication system with eight Boolean propositions (`LoginFailure`, `LoginSuccess`, `AccountLocked`, `AccountUnlocked`, `AdminVerified`, `PasswordResetRequested`, `UserNotified`, `SuspiciousActivityDetected`) and four real-valued signals (`AnomalyScore`, `FailedAttemptRate`, `LegitimateUserConfidence`, `AccountLockStatus`): (A) write LTL formulas for six requirements; (B) write MTL formulas for six timing requirements; (C) write STL formulas for six requirements on the real-valued signals with $A_{\text{crit}} = 0.8$, $F_{\text{crit}} = 5$, $C_{\text{safe}} = 0.9$; (D) answer three conceptual questions; (E) construct four example traces.

Solution: **Notation and assumptions.** $\Box$ is "always", $\Diamond$ is "eventually", $\bigcirc$ is "next", $\mathcal{U}$ is "until" and $\varphi \,\mathcal{W}\, \psi \equiv (\varphi \,\mathcal{U}\, \psi) \vee \Box\varphi$ is "weak until" (the same as until, except that ψ is allowed never to happen). In MTL and STL, $\Diamond_{[a,b]}\varphi$ means φ holds at some time in the window $[t+a,\, t+b]$ and $\Box_{[a,b]}\varphi$ means φ holds at all times in that window; time is measured in seconds. I treat $\mathsf{AccountUnlocked} \equiv \neg\mathsf{AccountLocked}$ and take $\mathsf{AdminVerified}$, $\mathsf{PasswordResetRequested}$ and $\mathsf{UserNotified}$ as events that hold at the instant they happen.

Several requirements are about something *becoming* true, such as an account *becoming* locked. Writing $\Box(\mathsf{AccountLocked} \to \dots)$ would repeat the obligation at every instant the account stays locked, which is stronger than what is asked. So for "p becomes true" I use the abbreviation

$$\mathrm{On}(p,\, \varphi) := (p \to \varphi) \;\wedge\; \Box\big(\neg p \to \bigcirc(p \to \varphi)\big),$$

which requires φ exactly at the instants where p switches from false to true (and at the start if p already holds there).

Part A: LTL specifications.

A1. $\Box\big(\mathsf{AccountLocked} \to \neg\mathsf{LoginSuccess}\big)$.
At every instant, if the account is locked then no successful login is happening. The implication is the standard way to say "whenever p, then not q".

A2. $\mathrm{On}(\mathsf{AccountLocked},\ \mathsf{AccountLocked}\ \mathcal{W}\ \mathsf{AdminVerified})$.
From the moment the account becomes locked, it must stay locked until admin verification happens, so unlocking before that is forbidden. I chose weak until because the requirement never promises that verification will happen. If it doesn't, the account just stays locked forever, which is still allowed.

A3. $\Box\big((\mathsf{AdminVerified} \wedge \mathsf{AccountLocked}) \to \Diamond\,\mathsf{AccountUnlocked}\big)$.
Whenever verification is completed while the account is locked, the account must eventually be unlocked. This is a liveness property, so it uses $\Diamond$ and says nothing about *when*.

A4. $\Box\big(\mathsf{PasswordResetRequested} \to \Diamond(\mathsf{AccountUnlocked} \vee \mathsf{AdminVerified})\big)$.
After any password-reset request, at least one of the two outcomes, unlocked account or completed verification, must eventually occur. The disjunction is inside the $\Diamond$ because either outcome is enough.

A5. $\Box\,\neg(\mathsf{LoginSuccess} \wedge \mathsf{LoginFailure})$.
This is a safety property: both can never be true at the same instant, which is what "never simultaneously" says.

A6. $\mathrm{On}(\mathsf{AccountLocked},\ \Diamond\,\mathsf{UserNotified})$.
Every time the account becomes locked, a notification must eventually follow (the notification may even happen at the same instant, since $\Diamond$ includes the present).

Part B: MTL specifications.

B1. $\mathrm{On}(\mathsf{SuspiciousActivityDetected},\ \Diamond_{[0,10]}\,\mathsf{AccountLocked})$.
The interval $[0, 10]$ turns the open-ended "eventually" into a deadline: the lock must happen within 10 seconds of suspicious activity starting, and the 0 allows it to be immediate.

B2. $\mathrm{On}(\mathsf{AccountLocked},\ \Diamond_{[0,30]}\,\mathsf{UserNotified}\ )$.
The user must be notified within 30 seconds after the account becomes locked. A notification after the 30-second window does not count.

B3. $\Box\big(\mathsf{PasswordResetRequested} \to \Diamond_{[0,600]}\,\mathsf{AdminVerified}\big)$.
10 minutes is 600 seconds, so verification has to be completed in $[0, 600]$ after the request.

B4. $\mathrm{On}(\mathsf{AccountLocked},\ \Box_{[0,60]}\mathsf{AccountLocked}\ \vee\ (\mathsf{AccountLocked}\ \mathcal{U}_{[0,60]}\ \mathsf{AdminVerified})\ )$.
After the lock starts, one of two things must hold. Either the account stays locked for the entire first 60 seconds ($\Box_{[0,60]}\mathsf{AccountLocked}$), or verification is completed at some time inside those 60 seconds and the account was locked until then ($\mathsf{AccountLocked}\,\mathcal{U}_{[0,60]}\,\mathsf{AdminVerified}$). In the second case the early unlock is excused, which covers the "unless verification has already been completed" part.

B5. $\Box\big(\mathsf{AdminVerified} \to \Diamond_{[0,120]}\,\mathsf{AccountUnlocked}\big)$.
Two minutes is 120 seconds, so the account must be unlocked within $[0, 120]$ after the verification is completed.

B6. $\Box\big(\mathsf{LoginFailure} \to \Diamond_{(0,20]}(\mathsf{LoginSuccess} \vee \mathsf{AccountLocked})\big)$.
The interval $(0, 20]$ looks strictly into the *following* 20 seconds (not the failure instant itself), and inside it at least one of the two events must occur, hence the disjunction. LoginSuccess stands for the legitimate login.

Part C: STL specifications. The atomic predicates are inequalities on the signals, and AccountLocked corresponds to $\mathsf{AccountLockStatus}(t) \geq 1$:

$$\mu_1 := (\mathsf{AnomalyScore}(t) < 0.8), \quad \mu_2 := (\mathsf{AnomalyScore}(t) > 0.8), \quad \mu_3 := (\mathsf{FailedAttemptRate}(t) > 5),$$
$$\mu_4 := (\mathsf{AccountLockStatus}(t) \geq 1), \quad \mu_5 := (\mathsf{AccountLockStatus}(t) < 1),$$
$$\mu_6 := (\mathsf{LegitimateUserConfidence}(t) > 0.9),$$
$$\mu_7 := (\mathsf{AnomalyScore}(t) < 0.3), \quad \mu_8 := (\mathsf{LegitimateUserConfidence}(t) < 0.5).$$

C1. $\Box\,\mu_1 \ =\ \Box\big(\mathsf{AnomalyScore}(t) < A_{\mathrm{crit}}\big)$.
The anomaly score is below the critical value at all times; during normal operation it never reaches 0.8.

C2. $\Box\big(\mu_2 \to \Diamond_{[0,10]}\,\mu_4\big)$.
Whenever the anomaly score exceeds 0.8, the lock status must reach 1 within 10 seconds. In plain terms, a high anomaly score must trigger a lock quickly.

C3. $\Box\big(\mu_3 \to \Diamond_{[0,5]}\,\mu_4\big)$.
If more than 5 failed attempts are being observed, the account must be locked within 5 seconds.

C4. $\Box\big((\mu_4 \wedge \mu_6 \wedge \mu_7) \to \Diamond_{[0,120]}\,\mu_5\big)$.
When the account is locked, the user is very likely legitimate ($\mathsf{LegitimateUserConfidence} > 0.9$) and the anomaly score is low (< 0.3), it must be unlocked within 2 minutes. This is the recovery rule for genuine users who got locked out.

C5. $\Box\big((\mu_2 \wedge \mu_8) \to \mu_4\big)$.
At every instant where the anomaly score is high and confidence in the user is low, the account is locked, so it cannot be unlocked while that risky combination lasts.

C6. $\Box\big(\Box_{[0,5]}\,\mu_2\ \to\ \Diamond_{[5,10]}\,\mu_4\big)$.
If the score stays above 0.8 for a full 5 seconds starting at t, then the lock must appear in the next 5 seconds, i.e. in $[t + 5,\, t + 10]$. A short spike does not trigger the requirement; only persistent suspicious behaviour does.

Part D: Conceptual questions.

D1. LTL only talks about the order of events: $\Diamond$AccountLocked is satisfied even if the lock arrives ten hours after the suspicious activity, because "eventually" has no deadline. The requirement here is a hard 10-second deadline, so we need real time. MTL puts an interval on the temporal operators, so $\Diamond_{[0,10]}$AccountLocked says exactly this.

D2. Signals such as AnomalyScore(t), FailedAttemptRate(t) and LegitimateUserConfidence(t) take values in the real numbers, and requirements on them are about thresholds ("above 0.8"). STL builds this in through predicates $f(x(t)) > c$ over the continuous signals, together with timed operators. It also gives a robustness value that measures *how much* a signal satisfies or violates a requirement (for example, how far the anomaly score stays below 0.8), which tells us more than a plain yes/no. With LTL or MTL we would first have to turn each signal into Booleans and lose that margin.

D3.

- $\square\big(\text{SuspiciousActivityDetected} \rightarrow \Diamond\, \text{AccountLocked}\big)$ is **LTL**. The account gets locked at some point after suspicious activity, with no time limit.
- $\square\big(\text{SuspiciousActivityDetected} \rightarrow \Diamond_{[0,10]}\text{AccountLocked}\big)$ is **MTL**. The requirement is still Boolean, but now the lock must come within 10 seconds.
- $\square\big((\text{AnomalyScore}(t) > 0.8) \rightarrow \Diamond_{[0,10]}(\text{AccountLockStatus}(t) = 1)\big)$ is **STL**. The atoms are no longer Boolean labels but conditions on real-valued signals (a threshold on the anomaly score and a value of the lock status), and the timing bound stays.

Going from LTL to MTL adds *quantitative time* (bounded intervals, deadlines and durations). Going from MTL to STL adds *real-valued signals* through predicates, and with them robustness semantics that say how well a requirement is satisfied.

Part E: Example traces.

E1. A trace satisfying all of A1–A6. Each column is one step; a proposition not listed is false. The last state repeats forever.

Step	True propositions
0	AccountUnlocked, LoginSuccess
1	AccountUnlocked, LoginFailure
2	AccountLocked, LoginFailure, SuspiciousActivityDetected
3	AccountLocked, UserNotified, PasswordResetRequested
4	AccountLocked, AdminVerified
5	AccountUnlocked
$6, 7, \ldots$	AccountUnlocked, LoginSuccess (repeated forever)

Checking each requirement:

- A1: the account is locked in steps 2–4, and none of these has LoginSuccess.
- A2: the lock starts at step 2 and lasts until AdminVerified holds at step 4; it is released only afterwards (step 5).
- A3: AdminVerified $\wedge$ AccountLocked holds at step 4, and the account is unlocked at step 5.
- A4: the reset request at step 3 is followed by AdminVerified at step 4.
- A5: no step contains both LoginSuccess and LoginFailure.
- A6: the lock starts at step 2 and the user is notified at step 3.

E2. A trace violating at least two of the LTL specifications.

Step	True propositions
0	AccountUnlocked
1	AccountLocked, SuspiciousActivityDetected
2	AccountLocked, LoginSuccess, LoginFailure
$3, 4, \ldots$	AccountUnlocked (repeated forever)

This trace violates four of the specifications:

- **A1**: at step 2 the account is locked and LoginSuccess is true.
- **A5**: at step 2 both LoginSuccess and LoginFailure hold together.
- **A2**: the account becomes locked at step 1, but at step 3 it is unlocked even though AdminVerified never occurred.
- **A6**: the account is locked at step 1 but UserNotified never becomes true.

(A3 and A4 hold trivially because AdminVerified and PasswordResetRequested never occur.)

E3. A timed trace satisfying all of B1–B6.

Time (s)	Event
0	LoginSuccess (account unlocked)
5	LoginFailure
8	LoginSuccess
20	LoginFailure
24	SuspiciousActivityDetected becomes true
28	AccountLocked becomes true
40	UserNotified
50	PasswordResetRequested
100	AdminVerified
130	AccountUnlocked (account unlocked)
140	LoginSuccess

- B1: suspicious activity starts at 24 s and the lock follows at 28 s, i.e. after $4 \leq 10$ s.
- B2: the lock starts at 28 s and the notification comes at 40 s, which is $12 \leq 30$ s later.
- B3: the reset request at 50 s is answered by verification at 100 s, well within 600 s.
- B4: the account is locked from 28 s to 130 s. The first 60 s of the lock run until 88 s, and the account stays locked throughout that window (AdminVerified only arrives at 100 s).
- B5: verification at 100 s is followed by unlocking at 130 s, i.e. after $30 \leq 120$ s.
- B6: the failure at 5 s is followed by a successful login at 8 s, and the failure at 20 s is followed by the lock at 28 s (both within 20 s).

E4. A timed trace that violates the "locked within 10 seconds" requirement.

Time (s)	Event
0	AccountUnlocked
12	SuspiciousActivityDetected becomes true
25	AccountLocked becomes true

- Suspicious activity is detected at $t = 12$ s.
- The account does become locked, but only at $t = 25$ s, which is 13 s later.
- The violated specification is B1, $\mathrm{On}(\mathsf{SuspiciousActivityDetected}, \Diamond_{[0,10]}\mathsf{AccountLocked})$: the window $[12, 22]$ contains no instant at which the account is locked, so the deadline is missed by 3 seconds.